

UNIVERSIDAD DE LOS ANDES



NATURAL LANGUAGE PROCESSING

ISIS 4221

Tarea 3

Autores:

Isabella MARTÍNEZ

Gustavo MENDEZ

Paula DAZA

Nicolás KLOPSTOCK

Octubre 07, 2024

Tabla de contenidos

1	Introducción	2
2	Clasificador para 20N	2
2.1	Preprocesamiento de datos	2
2.2	Tokenización y Normalización	2
2.3	Entendimiento de datos	2
2.4	Regresión Logística	3
2.4.1	CountVectorizer	3
2.4.2	Evaluación	4
2.4.3	TfidfVectorizer	4
2.4.4	Evaluación	5
2.5	Naïve Bayes	6
2.5.1	CountVectorizer	6
2.5.2	Evaluación	6
2.5.3	TfidfVectorizer	7
2.5.4	Evaluación	7
2.6	Cross-Validation	8
2.6.1	Cross-Validation para Regresión Logística	8
2.6.2	Crass-Validation para Naïve Bayes	9
3	Análisis de Sentimiento	9
3.1	Preprocesamiento de datos	9
3.2	Construcción de características de Lexicons	10
3.2.1	Creación de características	11
3.3	Entrenamiento y evaluación de modelos	12
3.3.1	Dominio más difícil de clasificar	13
3.3.2	Características más importantes	13
4	Un clasificador para todos los dominios	15
4.1	Entrenamiento y evaluación de modelos	15
4.1.1	Características más importantes	16
4.2	¿Vale la pena entrenar un clasificador por dominio?	17
5	Conclusión	17

1 Introducción

Esta tarea aborda dos temas principales. El primero está enfocado en el uso de algoritmos (Naïve Bayes y Regresión Logística) sobre el datasets 20Newsgroups (20N) para identificar diferentes grupo de noticias y evaluar los resultados usando métricas de clasificación. El segundo está relacionado con usar los mismos algoritmos para hacer análisis de sentimiento sobre el dataset Multi-Domain Sentiment Dataset y comparar los respectivos resultados.

2 Clasificador para 20N

En esta sección, se detalla el uso de los algoritmos Naïve Bayes y Regresión Logística sobre el conjunto de datos 20N. El proceso involucró la consolidación de documentos, tokenización, normalización, entendimiento y revisión de datos y, por último el entrenamiento de modelos y sus respectivas evaluaciones.

2.1 Preprocesamiento de datos

El primer paso fue preprocesar los datos del conjunto dado (20N) para poder, luego, tratarlos de la forma correcta. Para esto, se crea un DataFrame de pandas. Al momento de crear el dataframe, se limpian los textos para que queden estandarizados (sin símbolos y sin ciertas líneas ruidosas) usando un regex.

2.2 Tokenización y Normalización

A continuación, se realizó la tokenización de los textos. Para esto, el notebook implementa la clase DocumentPreprocessor, que toma los textos consolidados y los tokeniza por oración, asegurando las siguientes transformaciones:

- Normalización del texto (minúsculas, etc.).
- Lematización de tokens.
- Eliminación de stop words.

Este proceso permite una normalización de los datos, que es una forma más sencilla de tratarlos. La siguiente función muestra un fragmento condensado de cómo se implementó el preprocesamiento:

```
def preprocess_document(self, doc: str) -> str:
    tokens = word_tokenize(doc)
    tokens = [self.lemmatizer.lemmatize(token.lower()) for token in tokens
              if token.lower() not in self.stop_words and token.isalnum()]
    return ' '.join(tokens)
```

Esta función se aplica al archivo de datos original.

2.3 Entendimiento de datos

Con los datos cargados, pudimos ver la distribución de cada clase.

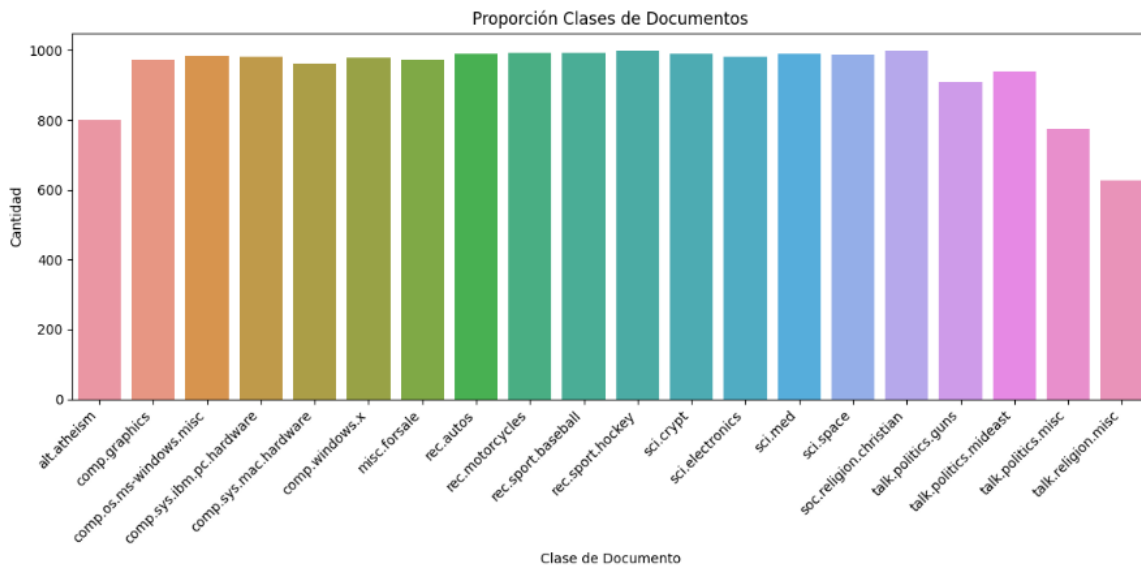


Figure 1: Distribución de clases en el dataset 20N

Podemos ver que, aunque clases como `talk.religion.misc` tengan una mayor diferencia de instancias, el resto de clases están muy bien balanceadas.

2.4 Regresión Logística

El primer algoritmo implementado fue *Regresión Logística*. Para este algoritmo, usamos dos implementaciones para el conteo de tokens: `CountVectorizer` y `TfidfVectorizer`.

2.4.1 CountVectorizer

Para este enfoque, usamos clases de la librería `sklearn`:

```
pipeline_c_lr = Pipeline([
    ('vectorizer', CountVectorizer()),
    ('classifier', LogisticRegression())
])

pipeline_c_lr.fit(X_train, y_train)
```

Con el modelo entrenado, podemos revisar las métricas de evaluación.

2.4.2 Evaluación

Reporte de Clasificación:				
	precision	recall	f1-score	support
alt.atheism	0.75	0.76	0.75	229
comp.graphics	0.68	0.76	0.72	295
comp.os.ms-windows.misc	0.73	0.73	0.73	289
comp.sys.ibm.pc.hardware	0.72	0.73	0.72	292
comp.sys.mac.hardware	0.77	0.79	0.78	256
comp.windows.x	0.83	0.77	0.80	319
misc.forsale	0.71	0.86	0.77	298
rec.autos	0.82	0.89	0.86	304
rec.motorcycles	0.94	0.88	0.91	326
rec.sport.baseball	0.86	0.93	0.90	294
rec.sport.hockey	0.97	0.90	0.93	300
sci.crypt	0.91	0.84	0.88	290
sci.electronics	0.78	0.77	0.77	321
sci.med	0.89	0.86	0.87	299
sci.space	0.87	0.86	0.87	288
soc.religion.christian	0.85	0.81	0.83	329
talk.politics.guns	0.76	0.80	0.78	233
talk.politics.mideast	0.93	0.89	0.91	280
talk.politics.misc	0.78	0.71	0.74	228
talk.religion.misc	0.64	0.56	0.60	179
accuracy			0.81	5649
macro avg	0.81	0.80	0.81	5649
weighted avg	0.82	0.81	0.81	5649

Figure 2: Reporte de clasificación por categoría (CountVectorizer)

Podemos resaltar las métricas:

- **Promedio** precision: 0.8095
- **Promedio** recall: 0.8050
- **Promedio** f1-score: 0.8060

2.4.3 TfidfVectorizer

Para este enfoque, usamos clases de la librería sklearn:

```
pipeline_tfidf_lr = Pipeline([
    ('vectorizer', TfidfVectorizer()),
    ('classifier', LogisticRegression())
])

pipeline_tfidf_lr.fit(X_train, y_train)
```

Con el modelo entrenado, podemos revisar las métricas de evaluación.

2.4.4 Evaluación

Reporte de Clasificación:				
	precision	recall	f1-score	support
alt.atheism	0.75	0.77	0.76	229
comp.graphics	0.75	0.80	0.78	295
comp.os.ms-windows.misc	0.72	0.76	0.74	289
comp.sys.ibm.pc.hardware	0.77	0.77	0.77	292
comp.sys.mac.hardware	0.82	0.80	0.81	256
comp.windows.x	0.84	0.80	0.82	319
misc.forsale	0.75	0.85	0.80	298
rec.autos	0.89	0.90	0.89	304
rec.motorcycles	0.95	0.93	0.94	326
rec.sport.baseball	0.94	0.95	0.95	294
rec.sport.hockey	0.97	0.96	0.97	300
sci.crypt	0.95	0.88	0.91	290
sci.electronics	0.80	0.82	0.81	321
sci.med	0.90	0.92	0.91	299
sci.space	0.89	0.91	0.90	288
soc.religion.christian	0.81	0.86	0.84	329
talk.politics.guns	0.80	0.92	0.86	233
talk.politics.mideast	0.95	0.91	0.93	280
talk.politics.misc	0.85	0.76	0.81	228
talk.religion.misc	0.77	0.47	0.58	179
accuracy			0.85	5649
macro avg	0.84	0.84	0.84	5649
weighted avg	0.85	0.85	0.84	5649

Figure 3: Reporte de clasificación por categoría (TfidfVectorizer)

Podemos resaltar las métricas:

- **Promedio** presicion: 0.8435
- **Promedio** recall: 0.8370
- **Promedio** f1-score: 0.8390

2.5 Naïve Bayes

El segundo algoritmo fue *Naïve Bayes*. Para este algoritmo, usamos ambas formas de conteo de tokens vistas en la sección anterior.

2.5.1 CountVectorizer

Para este enfoque, usamos clases de la librería `sklearn`:

```
pipeline_c_nb = Pipeline([
    ('vectorizer', CountVectorizer()),
    ('to_dense', transform_sparse), # transformamos la matriz dispersa de CountVectorizer a una densa
    ('classifier', GaussianNB())
])

pipeline_c_nb.fit(X_train, y_train)
```

Con el modelo entrenado, podemos revisar las métricas de evaluación.

2.5.2 Evaluación

Reporte de Clasificación:				
	precision	recall	f1-score	support
alt.atheism	0.71	0.75	0.73	229
comp.graphics	0.63	0.67	0.65	295
comp.os.ms-windows.misc	0.82	0.52	0.64	289
comp.sys.ibm.pc.hardware	0.57	0.73	0.64	292
comp.sys.mac.hardware	0.64	0.77	0.70	256
comp.windows.x	0.81	0.76	0.78	319
misc.forsale	0.77	0.55	0.64	298
rec.autos	0.86	0.80	0.83	304
rec.motorcycles	0.86	0.89	0.88	326
rec.sport.baseball	0.90	0.87	0.88	294
rec.sport.hockey	0.94	0.90	0.92	300
sci.crypt	0.79	0.88	0.83	290
sci.electronics	0.79	0.72	0.75	321
sci.med	0.92	0.86	0.89	299
sci.space	0.81	0.86	0.84	288
soc.religion.christian	0.89	0.81	0.85	329
talk.politics.guns	0.71	0.87	0.78	233
talk.politics.mideast	0.78	0.93	0.85	280
talk.politics.misc	0.72	0.79	0.75	228
talk.religion.misc	0.67	0.63	0.65	179
accuracy			0.78	5649
macro avg	0.78	0.78	0.77	5649
weighted avg	0.79	0.78	0.78	5649

Figure 4: Reporte de clasificación por categoría (CountVectorizer)

Podemos resaltar las métricas:

- **Promedio** precision: 0.7795
- **Promedio** recall: 0.7780
- **Promedio** f1-score: 0.7740

2.5.3 TfidfVectorizer

Para este enfoque, usamos clases de la librería sklearn:

```
pipeline_tfidf_nb = Pipeline([
    ('vectorizer', TfidfVectorizer(max_features=10000)),
    ('to_dense', transform_sparse),
    ('classifier', GaussianNB())
])
```

```
pipeline_tfidf_nb.fit(X_train, y_train)
```

Con el modelo entrenado, podemos revisar las métricas de evaluación.

2.5.4 Evaluación

Reporte de Clasificación:				
	precision	recall	f1-score	support
alt.atheism	0.64	0.59	0.62	229
comp.graphics	0.50	0.53	0.51	295
comp.os.ms-windows.misc	0.56	0.52	0.54	289
comp.sys.ibm.pc.hardware	0.53	0.54	0.53	292
comp.sys.mac.hardware	0.61	0.65	0.63	256
comp.windows.x	0.71	0.68	0.70	319
misc.forsale	0.65	0.52	0.58	298
rec.autos	0.78	0.69	0.73	304
rec.motorcycles	0.83	0.77	0.80	326
rec.sport.baseball	0.86	0.83	0.85	294
rec.sport.hockey	0.91	0.87	0.89	300
sci.crypt	0.75	0.86	0.80	290
sci.electronics	0.69	0.53	0.60	321
sci.med	0.84	0.77	0.80	299
sci.space	0.64	0.74	0.68	288
soc.religion.christian	0.79	0.71	0.75	329
talk.politics.guns	0.59	0.76	0.66	233
talk.politics.mideast	0.68	0.89	0.77	280
talk.politics.misc	0.54	0.68	0.60	228
talk.religion.misc	0.59	0.54	0.56	179
accuracy			0.69	5649
macro avg	0.68	0.68	0.68	5649
weighted avg	0.69	0.69	0.69	5649

Figure 5: Reporte de clasificación por categoría (TfidfVectorizer)

Podemos resaltar las métricas:

- **Promedio** precision: 0.6845
- **Promedio** recall: 0.6835
- **Promedio** f1-score: 0.6800

2.6 Cross-Validation

Cross-validation es un método usado para evaluar y garantizar que un modelo entrenado sea capaz de generalizar los datos no vistos, es decir, evitar over- y underfitting.

Este método se basa en separar los datos en varios conjuntos (*k-folds*). Luego, el modelo se entrena *k* veces. En cada una de las iteraciones, se toma uno de los subconjuntos como conjunto de test y el resto de conjuntos como training. Luego de realizar las *k* iteraciones, se calcula el promedio de las métricas de evaluación de iteración.

Este método tiene varias ventajas:

- Al usar varios conjuntos de evaluación, reduce la varianza de las estimaciones de rendimiento. Esto da una idea más precisa de cómo va a actuar el modelo con datos no conocidos.
- Como ya se mencionó, este método permite reducir (evitar, realmente) el sobreajuste. Esto, porque se evita que el modelo dependa de un mismo conjunto de datos, cuando se entrena con muchos subconjuntos.

2.6.1 Cross-Validation para Regresión Logística

Para este modelo específico se debe hacer una búsqueda de los hiperparámetros óptimos. Para esto, se define un esquema 10-fold para cross-validation. Esto quiere decir, dividir el conjunto de datos en 10 *folds* iguales. Luego, se usa la clase GridSearchCV de sklearn para hacer la búsqueda:

```
kf = KFold(n_splits=10, shuffle=True, random_state=42)

grid_search = GridSearchCV(pipeline_tfidf_lr, param_grid, cv=kf, verbose=2)
grid_search.fit(X_train_val, y_train_val)
```

Los mejores parámetros encontrados fueron {'classifier__C': 100, 'classifier__penalty': 'l2'}, donde classifier__C es el parámetro de regularización (penalización) (valores más bajos indican una regularización más fuerte) y classifier__penalty especifica el tipo de regularización a usar (l1 es Lasso y l2 es Ridge).

Además de esto, se encuentra la mejor precisión promedio obtenida en la cross-validación, en este caso **0.8677**.

Luego, podemos hacer una predicción con el conjunto de entrenamiento y validar usando el mejor modelo encontrado anteriormente. Los resultados dados son:

- Precision (Macro): 0.8741
- Recall (Macro): 0.8712
- F1-Score (Macro): 0.8723
- Precision (Micro): 0.8759
- Recall (Micro): 0.8759
- F1-Score (Micro): 0.8759

2.6.2 Crass-Validation para Naïve Bayes

Para este modelo, se usó la función `cross_val_predict` de la librería `sklearn` con la misma instancia `KFold` con un valor de 10 *folds*.

En este caso, las métricas de evaluación, prediciendo con el conjunto de prueba, dieron de la siguiente manera:

- Precision (Macro): 0.7962
- Recall (Macro): 0.7958
- F1-Score (Macro): 0.7933
- Precision (Micro): 0.7987
- Recall (Micro): 0.7987
- F1-Score (Micro): 0.7987

3 Análisis de Sentimiento

En esta sección, se detalla el proceso de implementación de análisis de sentimiento para el conjunto de datos Milti-Domain Sentiment Dataset. En particular, se quiere hacer un clasificador de sentimientos (positivo/negativo) para las categorías Books, DVD, Electronics y Kitchen.

3.1 Preprocesamiento de datos

Para cambiar la representación de los documentos, se implementó una función `representation_to_document`. Esta función convierte una representación del documento de un formato donde los términos se representan como parejas "palabra:conteo" en su forma plana y su etiqueta.

```
def representation_to_document(document_representation: str) -> Tuple[str, str]:
    word_counts = [term_representation.split(':') for term_representation in document_representation.split(' ')]

    _, label = word_counts[-1]

    word_counts = word_counts[:-1]

    words = []
    for term, count in word_counts:
        words.extend([term] * int(count))

    return ' '.join(words), label.replace("\n", "")
```

Luego, en la carga de datos y creación del DataFrame principal se separan explícitamente los documentos y su etiqueta respectiva.

3.2 Construcción de características de Lexicons

Como *Lexicon*, vamos a utilizar el proporcionado SentiWordNet. Con este, hay varias cosas a tener en cuenta para poder extraer las características.

Primero, vamos a eliminar los tokens <num>. Estos representan un valor numérico que no aporta nada al sentimiento positivo o negativo de la oración. Luego, manejamos palabras compuestas con guiones bajos y/o puntos. Estos se reemplazan por espacios, para obtener las palabras separadas y manejar cada una de manera individual.

```
def preprocess_word(word: str) -> List[str]:
    word = re.sub(r'<.*?>', '', word)

    word = word.replace('_', ' ').replace('.', ' ')

    words = word.split()

    return words
```

Otro aspecto que se tuvo que manejar fue, al momento de *parsear* el archivo del Lexicon, ignorar las líneas que empiezan con #.

Luego, extraemos los datos respectivos a PosScore, NegScore, SynsetTerms.

Es importante notar que, si una palabra tiene múltiples significados (p.e. *fire#1* y *fire#2*), se promedian sus puntajes positivos y negativos.

```
def load_sentiwordnet(path: Path) -> Dict[str, Dict[str, float]]:
    sentiwordnet = {}
    with open(path, 'r') as f:
        for line in f:
            # Ignoramos comentarios
            if line.startswith('#'):
                continue
            # La información está separada por tabs
            data = line.strip().split('\t')
            # Si una línea no tiene todos los datos, lo ignoramos
            if len(data) != 6:
                continue

            _, _, pos_score, neg_score, synset_terms, _ = data
            pos_score = float(pos_score)
            neg_score = float(neg_score)
            terms = synset_terms.split()

            # Vamos guardando el puntaje positivo y negativo por cada término
            for term in terms:
                word = term.split('#')[0]
                if word in sentiwordnet:
                    sentiwordnet[word]['pos_score'].append(pos_score)
                    sentiwordnet[word]['neg_score'].append(neg_score)
                else:
                    sentiwordnet[word] = {'pos_score': [pos_score], 'neg_score': [neg_score]}
```

```
# Promediamos los puntajes por palabra
for word in sentiwordnet:
    sentiwordnet[word]['pos_score'] = sum(sentiwordnet[word]['pos_score']) / \
    len(sentiwordnet[word]['pos_score'])
    sentiwordnet[word]['neg_score'] = sum(sentiwordnet[word]['neg_score']) / \
    len(sentiwordnet[word]['neg_score'])

return sentiwordnet
```

3.2.1 Creación de características

Para cada documento vamos a calcular las siguientes características:

1. Puntajes de sentimiento agregados:

- **Puntaje positivo total:** referido como `total_pos_score`. Suma de todos los `PosScore` de todas las palabras en el documento.
- **Puntaje negativo total:** referido como `total_neg_score`. Suma de todos los `NegScore` de todas las palabras en el documento.

2. Puntajes de sentimientos promediados:

- **Puntaje positivo promedio:** referido como `avg_pos_score`. Puntaje positivo total dividido entre el número de palabras.
- **Puntaje negativo promedio:** referido como `avg_neg_score`. Puntaje negativo total dividido entre el número de palabras.

3. Extremos y conteos:

- **Máximo puntaje positivo:** referido como `max_pos_score`. `PosScore` más alto de todas las palabras.
- **Máximo puntaje negativo:** referido como `max_neg_score`. `NegScore` más alto de todas las palabras.
- **Conteo de palabras positivas:** referido como `pos_word_count`. Número de palabras con `PosScore` mayor que 0.
- **Conteo de palabras negativas:** referido como `neg_word_count`. Número de palabras con `NegScore` mayor que 0.

4. Métricas combinadas:

- **Diferencia del puntaje de sentimiento:** referido como `score_difference`. Es la diferencia entre Puntaje positivo total y Puntaje negativo total.
- **Tasa del puntaje de sentimiento:** referido como `score_ratio`. Es Puntaje positivo total dividido entre Puntaje negativo total. Se manejan divisiones entre 0.

5. Nota:

- A las palabras que no se encuentran en el lexicon se les asigna puntajes neutrales.

3.3 Entrenamiento y evaluación de modelos

Se implementó la función `evaluate_domains`, la cual evalúa ambos modelos (Regresión Logística y Naïve Bayes, usando `CountVectorizer` y `TfidfVectorizer`) para cada dominio.

Podemos comparar las accuracies por modelo, y dentro de cada uno por dominio.

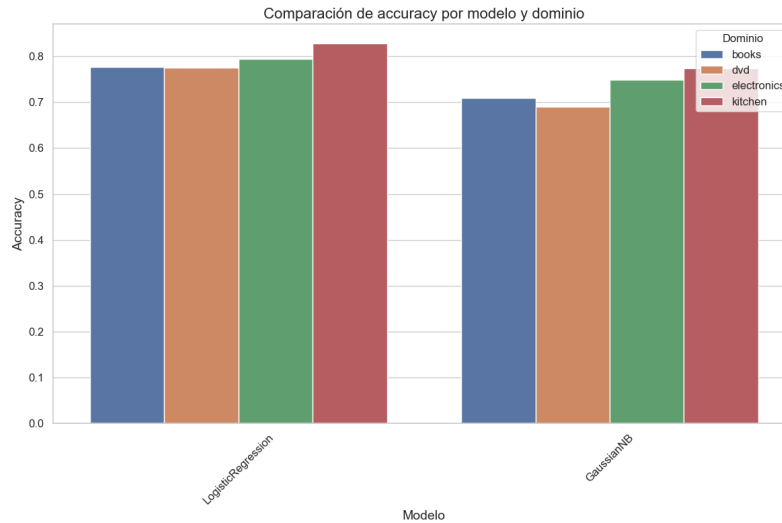


Figure 6: Comparación de accuracy por modelo y dominio

Se puede ver que la Regresión Lineal es la que mejor trabajo hace clasificando.

Ahora, se muestra una comparativa de accuracies según el vectorizador usado y el respectivo dominio.

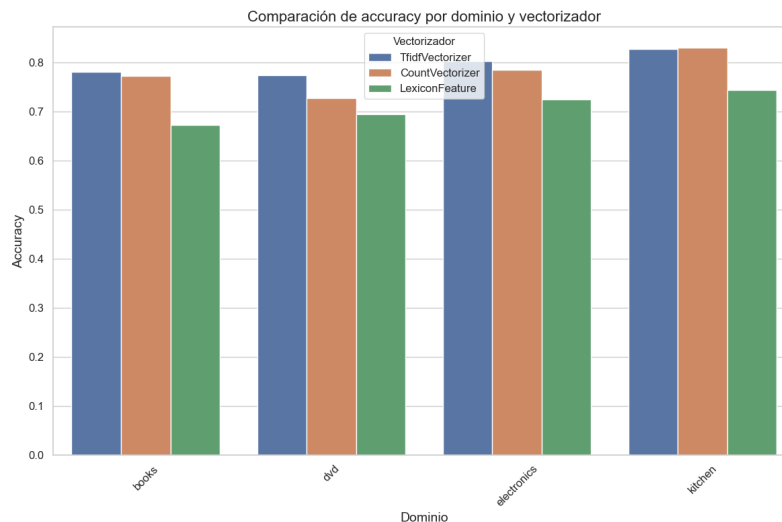


Figure 7: Comparación de accuracy por vectorizador y dominio

A excepción de una leve diferencia para el dominio Kitchen, la mejor representación es TF-IDF.

3.3.1 Dominio más difícil de clasificar

Para determinar cuál fue el dominio más difícil de clasificar, vamos a agrupar los datos por dominio y promediar sus accuracies. Vemos que DVD es el dominio que posee los peores resultados:

Domain	Accuracy
dvd	0.732571
books	0.742703
electronics	0.771372
kitchen	0.801149

Table 1: Accuracy por dominio

3.3.2 Características más importantes

A continuación, vamos a resaltar el top 10 de características más importantes para todos los modelos de Regresión Logística.

Dominio	Modelo	Vectorizador	Top Características (Feature: Coefficient)
books	LogisticRegression	CountVectorizer	truth_hurts: 0.0501, year_the: 0.0501, substance_only: -0.0444, book_no: -0.0440, sandford_is: 0.0432, a_mastermind: 0.0432, boring: -0.0422, fast_it: 0.0410, waste: -0.0398, hurts: 0.0394
	LogisticRegression	LexiconFeature	avg_neg_score: -0.8464, avg_pos_score: 0.5065, max_neg_score: -0.4222, max_pos_score: 0.3950, pos_word_count: -0.1289, neg_word_count: 0.1218, score_ratio: -0.0650, score_difference: 0.0177, total_neg_score: -0.0042, total_pos_score: 0.0020
	LogisticRegression	TfidfVectorizer	not: -0.0417, waste: -0.0407, excellent: 0.0388, boring: -0.0380, great: 0.0375, bad: -0.0357, too: -0.0343, a_must: 0.0325, disappointing: -0.0310, loved: 0.0302
dvd	LogisticRegression	CountVectorizer	or_heard: 0.0568, awful_film: -0.0457, awful: -0.0457, an_awful: -0.0457, super_and: 0.0447, waste: -0.0445, standard_elmo: 0.0414, elmo_: 0.0414, worst: -0.0403, tale_well: 0.0402
	LogisticRegression	LexiconFeature	avg_neg_score: -0.6690, neg_word_count: 0.3829, score_difference: 0.2718, max_neg_score: -0.2596, score_ratio: 0.2577, avg_pos_score: 0.2523, max_pos_score: 0.2136, total_neg_score: -0.1773, pos_word_count: -0.1123, total_pos_score: -0.0498
	LogisticRegression	TfidfVectorizer	great: 0.0509, not: -0.0435, love: 0.0413, bad: -0.0394, excellent: 0.0384, worst: -0.0382, waste: -0.0379, boring: -0.0361, i_love: 0.0354, best: 0.0343
electronics	LogisticRegression	CountVectorizer	great: 0.0821, cost_to: -0.0814, price_at: 0.0687, two_ends: -0.0661, ends_was: -0.0661, price: 0.0658, not: -0.0658, waste: -0.0588, great_product: 0.0583, not_work: -0.0555
	LogisticRegression	LexiconFeature	neg_word_count: 0.9836, avg_pos_score: 0.7013, score_difference: 0.6538, max_neg_score: -0.6111, total_neg_score: -0.5867, score_ratio: -0.2340, pos_word_count: -0.2236, avg_neg_score: -0.2224, max_pos_score: 0.2096, total_pos_score: -0.1962
	LogisticRegression	TfidfVectorizer	great: 0.0923, not: -0.0815, price: 0.0651, excellent: 0.0554, not_work: -0.0524, work: -0.0506, waste: -0.0487, works: 0.0485, the_price: 0.0453, good: 0.0450
kitchen	LogisticRegression	CountVectorizer	great: 0.0911, not: -0.0676, love: 0.0667, easy_to: 0.0659, easy: 0.0645, beautiful_a: 0.0643, disappointed: -0.0592, recommend_a: 0.0541, it_yet: -0.0519, product_have: -0.0518
	LogisticRegression	LexiconFeature	neg_word_count: 1.0310, avg_pos_score: 0.8918, max_neg_score: -0.5189, score_difference: 0.5063, avg_neg_score: -0.4412, pos_word_count: -0.3728, total_neg_score: -0.3620, max_pos_score: 0.0842, total_pos_score: -0.0723, score_ratio: -0.0319
	LogisticRegression	TfidfVectorizer	great: 0.0932, not: -0.0862, easy: 0.0669, love: 0.0663, easy_to: 0.0624, disappointed: -0.0569, best: 0.0517, a_great: 0.0494, i_love: 0.0482, works: 0.0452

Table 2: Top características por dominio, modelo y vectorizador

1. **Count Vectorizer:** Captura solo la frecuencia de aparición de palabras sencillas o compuestas. Las features más importantes suelen ser términos muy específicos y frecuentes dentro del dominio.

- En **books**, se destacan palabras o frases como *"boring"* y *"waste"*, lo que indica que el análisis está fuertemente ligado a expresiones comunes y negativas que suelen aparecer en las reseñas.

- En **dvd** y **electronics**, términos como *"awfull film"* y *"waste"* están asociados con reseñas negativas, mientras que *"great"* y *"price"* se asocian con reseñas positivas.
 - En **kitchen**, términos positivos como *"great"* y *"easy to"* se destacan, mientras que las reseñas negativas están marcadas por palabras como *"disappointed"*.
2. **TF-IDF**: Captura la relevancia de las palabras al reducir la importancia de aquellas que aparecen con demasiada frecuencia en todos los documentos, favoreciendo las que son más distintivas.
- En **books**, términos como *"excellent"* y *"great"* tienen un peso positivo, mientras que palabras negativas como *"waste"* y *"boring"* tienen un peso negativo.
 - En **dvd**, *"love"* y *"great"* son los términos más representativos de reseñas positivas, mientras que *"waste"* y *"boring"* se destacan en reseñas negativas.
 - En **electronics** y **kitchen**, términos como *"great"*, *"easy"*, y *"love"* aparecen fuertemente asociados a reseñas positivas, mientras que *"waste"*, *"not"*, y *"disappointed"* indican reseñas negativas.
3. **Lexicon based features**: Se basa en puntuaciones predefinidas de palabras según un léxico de sentimiento, en este caso *SentiWordNet*.
- En todos los dominios, la *"avg_neg_score"* y *"neg_word_count"* tienen una alta influencia en reseñas negativas, lo que resalta cómo este vectorizador mide las cargas emocionales negativas.
 - Características como *"score_ratio"* y *"avg_pos_score"* reflejan el balance entre palabras positivas y negativas, afectando la puntuación global de la reseña.

4 Un clasificador para todos los dominios

Es esta sección se va a explicar el proceso de implementación de un clasificador que clasifique para todos los dominios (Books, DVD, Electronics y Kitchen).

4.1 Entrenamiento y evaluación de modelos

Primero, se van a consolidar todos los dominios en un solo dataset de entrenamiento y validación, y crearemos clasificadores para estos datos consolidados.

Ahora, podemos comparar de nuevo accuracies según el modelo y vectorizador.

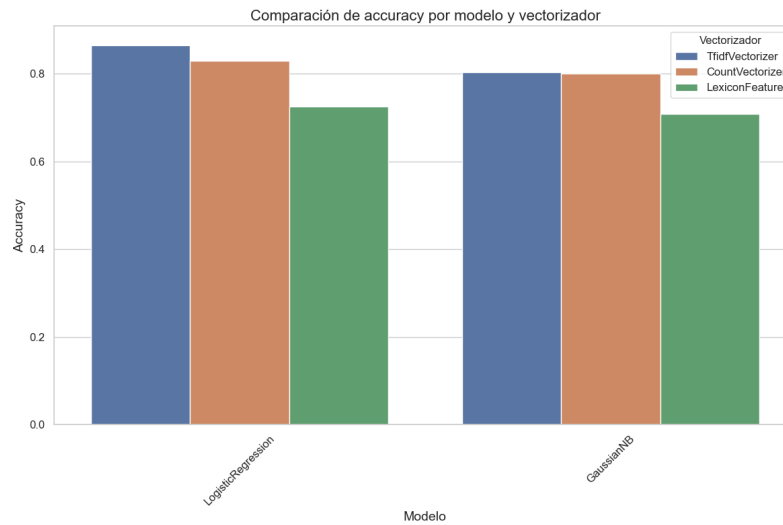


Figure 8: Comparación de accuracy por modelo y vectorizador

Se puede ver que, de nuevo, Regresión Logística es el modelo que mejor se comporta y obtiene mejores resultados, comparando con Naïve Bayes.

4.1.1 Características más importantes

Modelo	Vectorizador	Top Características (Feature: Coefficient)
LogisticRegression	CountVectorizer	great: 0.0911, waste: -0.0657, is_piece: -0.0649, not: -0.0617, great_product: 0.0602, price_at: 0.0593, excellent: 0.0576, easy_to: 0.0567, product_have: -0.0563, disappointed: -0.0535
	LexiconFeature	neg_word_count: 0.6986, avg_pos_score: 0.6758, avg_neg_score: -0.6632, max_neg_score: -0.4493, pos_word_count: -0.3203, score_difference: 0.2828, max_pos_score: 0.2400, total_neg_score: -0.2323, total_pos_score: -0.0999, score_ratio: -0.0934
	TfidfVectorizer	great: 0.0970, not: -0.0876, excellent: 0.0606, waste: -0.0566, easy: 0.0549, love: 0.0549, easy_to: 0.0533, disappointed: -0.0511, best: 0.0478, price: 0.0476

Table 3: Top características por modelo y vectorizador

1. Count Vectorizer:

- Términos como "great" y "excellent" tienen un coeficiente positivo, lo que sugiere que son características comunes en reseñas positivas.
- Palabras como "waste", "not", y "disappointed" tienen coeficientes negativos, lo que indica que se utilizan comúnmente en reseñas negativas.

2. TF-IDF:

- Palabras como "great", "excellent", y "easy" reciben coeficientes positivos significativos.

- Nuevamente, términos negativos como *"waste"*, *"not"*, y *"disappointed"* tienen coeficientes negativos.

3. Lexicon based features:

- El *"neg_word_count"* (cantidad de palabras negativas) tiene el coeficiente más alto, lo que indica que las reseñas con más palabras negativas tienen una mayor influencia en la predicción negativa del modelo.
- La *"avg_pos_score"* (puntuación promedio de palabras positivas) también es una característica importante, lo que implica que las reseñas con mayor promedio positivo tienen un impacto significativo en las predicciones.
- Las métricas como *"score_difference"* (diferencia de puntajes) y *"total_neg_score"* muestran cómo se pondera el sentimiento de una reseña de manera general.

En comparación con las características importantes obtenidas en el literal anterior, podemos destacar:

- En los clasificadores por dominio, como en **Count Vectorizer**, se capturan términos muy específicos que son relevantes solo para ese contexto:
 - En **books**, términos como *"truth_hurts"*, *"sandford.is"*, y *"book.no"* reflejan un vocabulario relacionado con reseñas literarias.
 - En **dvd**, frases como *"awfull_film"*, *"elmo_"*, y *"tale_well"* son términos típicos de reseñas de películas o entretenimiento.
 - En **electronics**, palabras como *"price"*, *"waste"*, y *"not_work"* reflejan preocupaciones específicas de ese dominio, como el costo y la funcionalidad del producto.
- Los clasificadores consolidados muestran características más generales:
 - Términos como *"great"*, *"waste"*, *"not"*, y *"excellent"* aparecen entre las principales características, las cuales son aplicables a múltiples dominios (libros, DVD, electrónicos, etc.).

4.2 ¿Vale la pena entrenar un clasificador por dominio?

Lo que identificamos es que el clasificador consolidado alcanza un accuracy mayor (0.865) en comparación con la mayoría de los obtenidos para los dominios específicos, que son:

- books: 0.839
- dvd: 0.827
- electronics: 0.854
- kitchen: 0.875

Solo el clasificador para kitchen es 1 punto porcentual superior, pero no supone una mejora significativa.

5 Conclusión

En conclusión, se pudo comprobar que los algoritmos de **Regresión Logística** superaron en rendimiento a **Naïve Bayes** tanto en la clasificación de noticias del dataset 20Newsgroups, como en la clasificación de sentimientos en el Multi-Domain Sentiment Dataset. En ambos casos, el uso de TfidfVectorizer como método de extracción de características mostró ser la estrategia más eficiente, proporcionando mejoras en métricas de precisión, recall

y F1-score. Por otro lado, si bien se evaluó la posibilidad de crear un clasificador individual por cada dominio, se concluyó que un clasificador consolidado para todos los dominios ofrecía resultados comparables, con una diferencia marginal en términos de precisión, lo que lo convierte en una opción más eficiente.

Finalmente, el análisis de las características más importantes dentro de los modelos de **Regresión Logística** confirmó que ciertas palabras tienen un gran impacto al momento de predecir sentimientos, destacando términos como "great" y "waste" como altamente representativos de reseñas positivas y negativas, respectivamente.